

# 一、实验目的

本实验目的是加强学生对位级运算的理解及熟练使用的能力。

# 二、报告要求

本报告要求学生把实验中实现的所有函数逐一进行分析说明，写出实现的依据，也就是推理过程，可以是一个简单的数学证明，也可以是代码分析，根据实现中你的想法不同而异。

# 三、函数分析

## 1. bitXor函数

函数要求：

| 函数名  | bitXor                |
|------|-----------------------|
| 参数   | int , int             |
| 功能实现 | x^y                   |
| 要求   | 只能使用 ~ 和   运算符，将结果返回。 |

实现分析：

我们使用De Morgan's laws对x^y进行变换。

x xor y = (x and noty) or (notx and y) =not(not(x and not y)与 not(not x and y)) 如此可利用and和not实现xor。

函数实现：

```
int bitXor(int x, int y) {
    return ~(~(x & (~y)))&(~((~x)&y));
}
```

## 2. getByte函数

函数要求：

| 函数名  | getByte     |
|------|-------------|
| 参数   | int, int    |
| 功能实现 | 从字x中取出第n个字节 |

函数分析：

我们的目的是取出某部分数据，所以最后一定是和0x000000FF进行与的操作。

在这之前，我们需要把x的特定字节右移到最低位，所以我们需要x>>(n乘8)

但是！我们没有办法使用乘法，使用八次加法超过限制，故要想到n乘8=n<<3，思路成立。

函数实现：

```
int getByte(int x, int n) {
    int y=0x000000FF;
    return (x>>(n<<3))&y;
}
```

## 3. logicalShift函数

函数要求：

| 函数名  | logicalShift     |
|------|------------------|
| 参数   | int , int        |
| 功能实现 | x>>n,locical     |
| 要求   | 实现逻辑右移而非自带的算术右移。 |

实现分析：

我们想到，逻辑右移与算术右移的区别在于前面n位。

所以利用and把后面32-n位取出来即可。

怎么取出来呢？我们将0x80000000也就是只有最高位是1的数右移n位再左移1位，然后取反，即可。

函数实现：

```
int logicalShift(int x, int n) {
    int y=0x01<<31;
    y=~((y>>n)<<1);
    return (x>>n)&y;
}
```

4. bitCount函数

函数要求：

| 函数名  | bitCount              |
|------|-----------------------|
| 参数   | int                   |
| 功能实现 | count how many 1 in x |

实现分析：

我们不能使用循环，所以必须分块考虑。

首先给出mask=8个连续的0001（2进制下），与x的移位依次做and，将32位分8块计算。

然后，对分块计算结果s，将前半部分加到后半部分，将后半部分的第1、3字节加到0、2字节上，然后再将0、2字节相加，与0b00111111取and，得到结果。

函数实现：

```
int bitCount(int x) {
    int mask=0x11 | (0x11<<8);
    int s=0x00;
    mask=mask|(mask<<16);
    s=x&mask;
    s+=(x>>1)&mask;
    s+=(x>>2)&mask;
    s+=(x>>3)&mask;
    s+=(s>>16);
    mask=0xF | (0xF<<8);
    s=(s&mask)+((s>>4)&mask);
    return (s+(s>>8))&0x3F;
}
```

5. conditional函数

函数要求：

| 函数名  | conditional                          |
|------|--------------------------------------|
| 参数   | int , int, int                       |
| 功能实现 | x?y:z                                |
| 要求   | if x==0, return z; if x!=0, return y |

实现分析：

我们需要考虑选择，即当x=0时，我们要输出z，否则输出y。

考虑掩码0x00000000（当x=1时）、0xFFFFFFFF（当x=0时），那么输出就应该是：(~mask & y) | (mask & z)。

那么如何形成掩码呢？利用!!x将x归为0或1，然后加上0xFFFFFFFF即可。

函数实现：

```
int conditional(int x, int y, int z) {
    int mask=!!x +(0xFF<<24>>24); //if x=0,mask=0xFFF...;if x!=0,mask=0
    return (~mask & y)|(mask & z);
}
```

## 6. tmin函数

函数要求：

| 函数名  | tmin                    |
|------|-------------------------|
| 参数   | void                    |
| 功能实现 | return the smallest int |

实现分析：

我们直接将0x01左移31位即可。

函数实现：

```
int tmin(void) {
    return 0x01<<31;
}
```

## 7. fitsBits函数

函数要求：

| 函数名  | fitsBits        |
|------|-----------------|
| 参数   | int , int       |
| 功能实现 | 查询x的补码是否能表示成n位。 |
| 要求   | 如果可以，输出1；否则输出0。 |

实现分析：

我们知道，能够表示成n位意味着n位之前要么全0要么全1。

那么，我们可以将n左移32-n位（这里我们使用32+~n+1计算32-n），再右移同样位数，如果与x相同则正确。

那么我们将移位之后的y与x异或，然后取反即可输出。

函数实现：

```
int fitsBits(int x, int n) {
    int z=33+~n;
    int y=x<<z;
    y=y>>z;
    return !(x^y);
}
```

## 8. dividePower2函数

函数要求：

| 函数名  | dividePower2       |
|------|--------------------|
| 参数   | int, int           |
| 功能实现 | compute x/pow(2,n) |
| 要求   | round toward 0     |

实现分析：

直接使用右移操作即可。但是我们需要向0舍入，所以当x为负数时，我们需要增加偏置。

对于x是负数的判断，使用掩码，掩码为x>>31，即如果x符号为1则由于算术右移，得到1...1。

我们需要加的是 $2^n-1$ ，所以表示为 $(1 \ll n)-1=(1 \ll n)+0xFFFFFFFF$ 。再进行计算即可。

函数实现：

```
int dividePower2(int x, int n) {
    int y=(x>>31)&((1<<n)+(0xFF<<24>>24));
    return (x+y)>>n;
}
```

9. negate函数

函数要求：

| 函数名  | negate         |
|------|----------------|
| 参数   | int            |
| 功能实现 | return -x      |
| 要求   | cannot use "-" |

实现分析：

我们知道对于补码，按位取反再+1即可实现取相反数。

函数实现：

```
int negate(int x) {
    return ~x+1;
}
```

10. howManyBits函数

函数要求：

| 函数名  | howManyBits  |
|------|--------------|
| 参数   | int          |
| 功能实现 | 计算表达x所需的最少位数 |

实现分析：

首先，我们将负数转化成正数进行统一处理。

这里我们不使用按位取反再+1，原因是对于 $1000\dots0$ ，其按位取反为自己。所以我们只进行按位取反操作。我们注意到，所有的负数与其按位取反得到的非负数的最低位数是一致的。

这里我们使用 $(x>>31)$ 异或 $x$ 来得到非负数。这是因为若 $x$ 为正，得到0异或 $x=x$ ；若为负，得到 $0xF\dots F$ 异或 $x=\sim x$ 。

接下来，我们只需要考虑非负数的情况。最低位实际上是最高1所在的位置+1。所以我们就是要找到最高的1。

我们采用二分法查找。对于一开始，令 $a16=!(x>>16)<<4$ ，这是因为是否在前16位是由 $x$ 右移16位后归一化得到0、1，得到是否至少16位。同样的用于后续 $a8$ 、 $a4$ 、 $a2$ 、 $a1$ 、 $a0$ 。将这些相加再加一得到我们的答案。

函数实现：

```

int howManyBits(int x) {
    int a16,a8,a4,a2,a1,a0;
    x=(x>>31)^x;
    a16=!!(x>>16)<<4;
    x=x>>a16;
    a8=!!(x>>8)<<3;
    x=x>>a8;
    a4=!!(x>>4)<<2;
    x=x>>a4;
    a2=!!(x>>2)<<1;
    x=x>>a2;
    a1=!!(x>>1);
    x=x>>a1;
    a0=x;
    return a16+a8+a4+a2+a1+a0+1;
}

```

## 11. isLessOrEqual函数

函数要求：

| 函数名  | isLessOrEqual           |
|------|-------------------------|
| 参数   | int, int                |
| 功能实现 | if x<=y, then return 1. |

实现分析：

我们希望使用y-x是否大于等于0来进行判断。但是，由于可能发生的溢出，我们需要先做一定的处理。

我们考虑y与x的符号是否不同。

如果：x为负，y为正，则直接正确。

如果xy同号，则计算y-x也就是y+(~x+1)并判断符号位即可得到结果。

判断正负只需要把数右移31位即可。

函数实现：

```

int isLessOrEqual(int x, int y) {
    int ifyzhengxfu=(~(y>>31)&(x>>31));//if correct, -1
    int samesign=~((y>>31)^(x>>31));//if correct, -1
    int ifybiggerx=~((y+~x+1)>>31);//if correct, -1
    return (ifyzhengxfu | (samesign & ifybiggerx))&0x01;
}

```

## 12. intLog2函数

函数要求：

| 函数名  | intLog2                 |
|------|-------------------------|
| 参数   | int                     |
| 功能实现 | return(log base 2 of x) |

实现分析：

即：查找最高位1的位置，然后-1。

我们在前面howManyBits函数中已经写过二分查找最高1位置的程序，这里直接套用。

我们采用二分法查找。对于一开始，令a16=!!(x>>16)<<4，这是因为是否在前16位是由x右移16位后归一化得到0、1，得到是否至少16位。同样的用于后续a8、a4、a2、a1、a0。

由于输入必定大于0，所以a0必为1，与后续-1相抵。所以最后将a16一直加到a1即可。

函数实现：

```

int intLog2(int x) {
    int a16,a8,a4,a2,a1;
    a16=!!(x>>16)<<4;
    x=x>>a16;
    a8=!!(x>>8)<<3;
    x=x>>a8;
    a4=!!(x>>4)<<2;
    x=x>>a4;
    a2=!!(x>>2)<<1;
    x=x>>a2;
    a1=!!(x>>1);
    return a16+a8+a4+a2+a1;
}

```

### 13. floatAbsVal函数

函数要求：

| 函数名  | floatAbsVal            |
|------|------------------------|
| 参数   | unsigned(a float)      |
| 功能实现 | return abs(f)          |
| 要求   | can use    && if while |

实现分析：

这里要求我们给出浮点数的绝对值。由于浮点数有专门的符号位并缺采用的是类似于原码的符号方式，故我们把符号为改0即可完成大部分任务。

我们需要读出阶码和尾数进行判断，如果是NaN的话直接输出其本身。

函数实现：

```

unsigned floatAbsVal(unsigned uf){
    unsigned exp=(uf>>23)&0xFF;
    unsigned frac=uf&0x7FFFFFFF;
    if(exp==0xFF && frac!=0)
    {
        return uf;
    }
    return uf&0x7FFFFFFF;
}

```

### 14. floatScale1d2函数

函数要求：

| 函数名  | floatScale1d2           |
|------|-------------------------|
| 参数   | int, int                |
| 功能实现 | if x<=y, then return 1. |
| 要求   | can use    && if while  |

实现分析：

简单的来说，计算0.5uf只需要将阶码-1即可，但很显然我们还需要考虑全面。

对于阶码全1的数（NaN或者inf），我们要返回其本身。

对于阶码全0的非规格化数，我们需要将尾数后移一位。

对于阶码为1的规格化数，乘0.5后将变为非规格化数，所以要为其把消失的1补上，并且对尾数进行右移。

对于其他规格化数，直接让阶码减1即可。

对于要右移尾数的两种情况，我们需要向偶舍入。

所以，首先取出符号、阶码、尾数三部分，添加两个判断值roundbit和roundup。roundbit表示尾数的最后一位，roundup表示倒数第二位。如果roundbit和roundup同时为1，我们需要为尾数+1后再进行右移，其他情况直接右移截断即可。

对阶码进行判断分类，对不同的阶码采取不同的处理方式即可完成任务。

函数实现：

```
unsigned floatScale1d2(unsigned uf) {
    unsigned sign=uf&(0x01<<31);
    unsigned exp=(uf>>23)&0xFF;
    unsigned frac=uf&0x7FFFFFFF;
    unsigned roundbit=frac&0x01;
    unsigned roundup=(frac>>1)&0x01;
    if(exp==0xFF)
    {
        return uf;
    }
    if(exp==0x00)
    {
        if(roundbit && roundup)
        {
            frac+=1;
        }
        frac=frac>>1;
        return sign|exp|frac;
    }
    if(exp==0x01)
    {
        frac=frac|(0x1<<23);
        if(roundbit && roundup)
        {
            frac+=1;
        }
        frac=frac>>1;
        exp=0x00;
        return sign|exp|frac;
    }
    exp=exp-1;
    exp=exp<<23;
    return sign|exp|frac;
}
```

15. floatFloat2Int函数

函数要求：

| 函数名  | floatFloat2Int         |
|------|------------------------|
| 参数   | unsigned               |
| 功能实现 | return (int)uf         |
| 要求   | can use    && if while |

实现分析：

我们希望把f转换为int。思考：f的尾数部分为23位，int表示32位。

所以：对于NaN、inf、实际阶码（去掉偏置之后的）大于等于31的情况，我们都不能表示。对于实际阶码小于0的，直接输出0。

对于其他范围内的阶码，取尾数、补最高1后判断实际阶码与23的大小进行左移、右移操作。最后，对于符号位为1的，取为负数即可返回。

函数实现：

```
int floatFloat2Int(unsigned uf) {
    unsigned sign=uf&(0x01<<31);
    unsigned exp=(uf>>23)&0xFF;
    unsigned frac=uf&0x7FFFFF;
    int E=exp-127;
    int newfrac=frac|(0x1<<23);
    if(exp==0xFF || E>=31)
    {
        return 0x80000000u;
    }
    if(E<0)
    {
        return 0;
    }
    if(E>23)
    {
        newfrac=newfrac<<(E-23);
    }
    else
    {
        newfrac=newfrac>>(23-E);
    }
    if(sign)
    {
        newfrac=-newfrac;
    }
    return newfrac;
}
```

## 四、实验总结

在本段总结实验中遇到的问题或困难，以及对应的解决方法；对实验的建议等。

本次实验不算是特涅特别难，但是令人感到十分痛苦。

实验过程中，运行dlc后告诉我我的代码中有些临时变量声明不成功（但运行程序正常），十分令人困惑。上网查询相关内容后发现，其使用的cpp标准较旧，要求所有临时变量的声明必须在非声明语句之前。更改后声明正常，dlc不再报错。

在实验过程中代码的问题通过运行./btest观察输入和期望的输出可以发现问题并作出调整。

那为什么说痛苦呢.....这样一个作业如果在我自己的电脑上做的话根本用不了十个小时（现在看我的在线时间貌似是八个小时左右）。其令人崩溃的没有中文输入但还要用中文写实验报告以及卡卡的软件、无法输入的bug、不能使用的快捷键都让人浪费许多无用的光阴在像我现在这样在外面剪切板里写中文.....

期望下次能够把实验报告独立出来，至少不用在网站上写。如计算机图形学提供LaTex模版、传感器技术与应用在在线笔记平台“语雀”上提供markdown模版、数据库系统提供docx模版等。